

Software processes for a changing world

Kevin T. Ryan^{*,†}

Lero – the Irish Software Engineering Research Centre, University of Limerick, Limerick, Ireland

ABSTRACT

Three key drivers of change in the world of software are identified, and their impact on a range of software engineering research is assessed. The rise of agile approaches to software development adds further pressure for software process research to develop and evolve processes that are supportive of change while meeting the most stringent needs of society. Two examples of successful process evolution, aimed at highly regulated industries, are outlined as pointers to the future direction of Software Process Research. [‡]

Copyright © 2015 John Wiley & Sons, Ltd.

Received 22 August 2015; Accepted 16 October 2015

KEY WORDS: process evolution; agile methods; regulated industry

1. INTRODUCTION

The recent history of industry and society has been one of deep and accelerating change. Much of that change has been driven by software so it may be surprising that software itself has proven difficult to change. We might imagine that, because software is ‘soft’ in comparison with the physical hardware it runs on, modifying software should be relatively easy. But we have learned to our cost that even the smallest change to software that has been running without a problem for months or years can lead to catastrophic failure. Most often, software is ‘frozen’ once it has been thoroughly tested before and after deployment, and a new, modified version will be developed off-line and only put into use once it too has been meticulously tested. In the modern world, software is often the key enabler of business and industrial processes, and it supports the constant innovation and improvement that are central to commercial success, but, paradoxically, the software that drives this change is not itself easily modified when that is needed. The processes adopted to ensure software quality have, if anything, been even slower to evolve. If we are to meet the ever-increasing expectations of society and commerce, we will have to do better. Somehow, we must devise processes that help us meet stringent quality standards while being flexible and responsive in the face of constant change.

The rest of this paper is organised as follows: Section 2 describes three major drivers of change in the world of software development and then outlines the added complication posed by the rapid spread of so-called ‘agile’ methods. Section 3 outlines the need for more flexibility in defining new processes and gives two examples of where this has been performed in collaboration with industry. Section 4 draws some tentative conclusions.

^{*}Correspondence to: Kevin T. Ryan, Lero – the Irish Software Engineering Research Centre, University of Limerick, Limerick, Ireland.

[†]E-mail: kevin.ryan@lero.ie

[‡]This paper is a revised and expanded version of a keynote abstract published by ACM in the proceedings of ICSSP 2014.

2. THE CHANGING WORLD OF SOFTWARE

Three mutually reinforcing external drivers have contributed greatly to the recent rapid change in the world of software. Specialisation, industrialisation and globalisation have disrupted and are continuing to refactor the world of software development to a degree that was anticipated by few, if any, researchers. More recently, agile methods have become a potent force for change arising from within the world of software practice, and their rapid spread has added further to the need for new and more flexible software processes.

2.1. Specialisation

Even though specialisation in software can be traced back to the very early days of computing when, for example, the distinction was first made between systems and applications programming, the notion of a general-purpose programmer survived well into the 1980s until the rapid dissemination of personal computers made it impossible to be expert in all their possible uses. The spread of software applications into almost every arena of human endeavour has spawned numerous areas of specialisation where the functionality and quality of the software are often the key determinant of competitive advantage. This is manifested in sectors such as smart phones, automobiles and passenger aircraft, where much of the market-oriented innovation is implemented in software. But software is increasingly the key differentiator in fields as diverse as medical devices, home entertainment and financial services and is increasingly influential in areas that are not considered software-based by the general public, for example, in agriculture and fisheries. The peculiarities of each of these domains influence the software development process and products. When systems failures could lead to loss of life or property, there is likely to be a requirement for systematic development, extensive documentation and, in many cases, conformance to set standards and independent certification. In response, software developers have devised models and methods to accommodate the domain specific requirements, and the software development processes have also to be changed or elaborated. An example of such process tailoring is the work of McCaffrey *et al.* [1] to reconcile the process steps of CMMI with those mandated by the (US) Food and Drugs Administration.

Specialisation has arisen in other dimensions also. The level of criticality, mentioned earlier, leads to differing approaches while large, medium and small organisations also have differing process needs.

2.2. Industrialisation

Cusamano [2] has argued that ‘software has the power to change the world’ but only if it is ‘treated as a business’ with all that implies. Software is now a key strategic competence for most large organisations, although some have been slow to accept this. Most modern products come from industrialised processes so why not software? There is a long-running debate about the need for, and current distance from, true software engineering, but, in any case, many engineering capabilities have been adopted in software development. Project management, version control, cost estimation and planned re-use are now widely used. An industrial example of the last named is to be seen in the development and use of software product lines (SPL) [3]. This is especially applicable in safety critical domains such as automotive, consumer appliances and mobile telephones. In these domains, we can anticipate the demand for a large but predictable set of variations on a standard version. In effect, we plan for what Parnas [4] has called a ‘family’ of software systems. SPL allows us to build in controlled flexibility in a safe and economic way. A more recent development is to support change at run-time by allowing systems to reconfigure themselves in response to (a limited set of) environmental changes [5]. This can be seen a first step toward highly reliable autonomic systems.

2.3. Globalisation

The massive growth in interconnection, economic, social, political and, especially, technological, has caused what we now call globalisation and, notwithstanding the uneven spread of connectivity, ever tighter connections seem inevitable. Software engineering is both a vehicle for and an early adopter

of globalisation. The motivations are obvious. Global Software Engineering (GSE) promises many benefits. Access to lower cost or better quality developers, ‘round the clock’ development as world-wide teams pass work on to teams whose day is only starting; more rapid responses to requests for correction or improvement, all of which should lead to lower cost and higher customer satisfaction. But experience has shown that it is difficult to realise these benefits, and some have argued that they often do not exist [6]. From the perspective of software processes, GSE poses specific challenges because teams are often separated by time, location, language and culture. Studies of GSE have sought to identify the key success factors and pinpoint the undoubted risks associated with its adoption [7]. One promising approach to mitigating these risks is to devise a process framework, in the style of CMMI, to support what is termed ‘Global Teaming’ [8].

2.4. *The rise of agile*

The greatest challenge to conventional software process research has been the unexpectedly rapid and widespread adoption of so-called ‘agile’ development methods. While we can dispute some of the more extreme claims of agile enthusiasts, the move from a staged to an incremental model of development has been almost universal and largely successful. Not every item on the various agile menus has been equally popular, and some are rarely adopted in practice, but the evidence is that the move to agile is beneficial and is now irreversible. A study by Forrester in 2009 reported 56% of developers using agile while one by Gartner in 2012 indicated this had risen to at least 84%. Also in 2012, the Standish Group published the CHOAS manifesto, which reported that, whereas only 14% of projects following the waterfall model were successful, this rose to 42% for those using agile approaches. All of these figures are open to challenge due to difficulties of definition, self-reporting and domain coverage, but the overall message is clear. Agile methods improve quality and are here to stay.

3. EVOLVING SOFTWARE PROCESSES

3.1. *Evolve or die*

The drivers of change identified earlier pose major challenges for software process research. In addition, we must learn to cope with the constant rapid change to the software itself, software that the world increasingly depends on. No longer can we simply freeze the software (and its requirements) while we develop, and exhaustively test, a new version of the software. In the modern environment, new services, functionality or performance requirements must be provided through safe and effective changes to the software all through its lifecycle. Lero – the Irish Software Engineering Research Centre (www.lero.ie) has a focus on ‘Evolving Critical Systems’, developing methods and tools that can be employed at design time or at run time to support efficient and safe modifications to important software systems. An example of the former, based on the technology mentioned in the section on Industrialisation, is studies of the SPL process [9]. An example of the latter is Lero’s work on high-reliability autonomous systems [10]. At a meta-level, Lero researchers were among the first to identify the need for ‘continuous’ software engineering [11].

3.2. *Software process research*

In retrospect, it is clear that software process research was ill-prepared for the rapid changes of the past 20 years. Going back to the origins of SPI, we can understand why this would be so. The pioneering work of Humphries at the SEI [12] arose within the context of very large, multi-year projects where documentation, legal liability and cost-plus contracts were the norm. Change requests, when they came, would be batched and prioritised before being incorporated into major new releases. Quite understandably, the original CMM reflected the traditional waterfall model, although subsequent variants have moved away from that. CMMI and other process frameworks have been extended, adapted or abbreviated to deal with Specialisation, Industrialisation and Globalisation, for example, in the work of Richardson *et al.* cited earlier.

It would be fair to say also that the rapid spread of agile methods was neither anticipated nor facilitated by the SPI community. At times, there has been an antagonistic relationship between the ‘old school’ of documentation-heavy CMM or ISO and the ‘new radicals’ of the agile community who show little regard for such concerns. Early arguments centred on whether agile was only suited to small, co-located teams. Extensive research and practical experience have challenged, if not refuted, those concerns. The limitations of the agile approaches seem to be at their greatest when we consider the stringent requirements for high-quality software, most especially in regulated domains. For example, following an EU medical directive in 2010, software can now be classified as a ‘medical device’ in its own right and be subject to all the regulation that implies. Such demands for provable quality do not, at first glance, sit comfortably with the agile priorities. After all, the agile manifesto emphasises ‘working software’ over ‘documentation’, ‘customer collaboration’ over ‘contract negotiation’ and ‘responding to change’ over ‘following a plan’. Most crucially, agile values ‘individuals and interactions’ over ‘processes and tools’. It was therefore a worthwhile challenge for Lero researchers, with some of their industrial partners, to investigate how agile approaches could be adapted for use in highly regulated sectors. Two examples are outlined here.

In the first, Fitzgerald *et al.* [13] worked with Qumas, a leading provider of compliance and quality management solutions in the Life Sciences. Qumas were moving away from their historical reliance on a waterfall model and wished to adopt agile approaches while maintaining the certification that was essential to their success in their sector. The work resulted in ‘R-Scrum’, a version of the popular agile approach that was enhanced to suit the stringent needs of compliance software in the life sciences domain. Repeated, small-scale quality assurance processes were incorporated into the traditional Scrum cycle. New roles were defined tasked with independent testing and documentation. A final ‘hardening sprint’ was added before the product would be released. It is doubtful if agile purists would approve of these variations, but the company quickly saw important benefits such as shorter release times and continuous traceability. A key conclusion for software researchers was that software in regulated industries has two customers: the end-user, for whom sales and marketing staff are often the proxy, and the regulator, for whom the firm’s quality assurance staff are the proxy.

In the second example by McHugh *et al.* [14] is also empirically based but is taking a bottom-up approach. Mandated standards in the medical device industry have traditionally reflected a waterfall philosophy, even though they are officially model neutral. The team aims to develop a hybrid life cycle model that embeds agile practices in the widely used V-model. A survey of four agile methods identified 59 candidate practices. Of these, 13 were considered to be easy to incorporate into the V-model without breaching the relevant standards. Examples of these include pair-programming, on-site customer and continuous integration. Work is continuing to extend, deploy and validate this model, but it is worth noting that industry standards change also, as exemplified by the publication of guidelines for using agile in medical device software [15].

4. CONCLUSIONS

What are the implications of all this for future SPI research? We could ask ourselves what is the high-level goal of all software process research? Is it to describe, in complete detail, an ideal process? Is it to help improve processes in themselves? Is it even to aid in the de-skilling of software developers? Surely, it is none or these but exists solely to support the production of better software-mediated products and services. This highlights the fundamental fact that products and services are what society needs. No one buys software for its own sake. Software is just a means to an end. A development process, no matter how elaborate or widely promoted, is only an indirect contributor to meeting society’s goals. Software process researchers need to get closer to the product, its context and its customers. Only then can they devise the flexible, easily tailored and continually evolving processes that will be needed so that we can be as agile in our process modelling as we are, increasingly, in our software development.

It is worth remarking that processes themselves depend on skilled people to enact them. Processes would only ‘be programs’ if people were computers. Bill Curtis is reputed to have said that the purpose of all SPI was ‘to support good people to do good work’. By this means, we build competence in

organisations and provide guidance for both novice and experienced developers. We cannot and should not try to automate the work of creative people.

The radical changes in SE and SPI of the past decade are only the first wave of many. The research community must devise software systems and development processes that deliver flexible and reliable systems that can evolve and improve over time without putting our societies, our citizens or our economies at risk.

ACKNOWLEDGEMENTS

This paper benefited from comments by Ita Richardson and Brian Fitzgerald on earlier versions, but any errors in representing their work are my own. Supported, in part, by Science Foundation Ireland CSET grant 10/CE/I1855 to Lero -the Irish Software Engineering Research Centre.

REFERENCES

1. McCaffery F, Coleman G. The need for a software process improvement model for the medical device industry. *International Review on Computers and Software (IRECOS)* 2007; **2**(1):10–15.
2. Cusumano M. *The Business of Software*. Free Press: New York, 2004.
3. Pohl K, Böckle G, van der Linden FJ. *SoAware Product Line Engineering*. Springer: Berlin, 2005.
4. Parnas DL. On the design and development of program families. *IEEE Transactions on Software Engineering* 1976; **2**(1):1–9.
5. Hallsteinsen S, Hinchey M, Park S, Schmid K. Dynamic soEware product lines. *IEEE Computer*, 2008.
6. Ó Conchúir E, Holmstrom Olsson H, Agerfalk P, Fitzgerald B. Global software development: never mind the problems – are there really any benefits? 29th IS Research Seminar in Scandinavia, 2006.
7. Richardson I, Casey V, McCaffery F, Burton J, Beecham S. A process framework for global software engineering teams. *Information and Software Technology* 2012; **54**(11):1175–1191.
8. Richardson I, Casey V, Burton J, McCaffery F. Global software engineering: a software process approach. *Collaborative Software Engineering* Mistrik I, Grundy J, Hoek A, van der Whitehead J (eds.). 2010; 35–56. ISBN:978-3-642-10293-6.
9. O’Leary P, Santana de Almeida E, Richardson I. The pro-PD process model for product derivation within software product lines. *Information and Software Technology* 2012; **54**:1014–1028.
10. Vassef E, Hinchey M. The ASSL formalism for real-time autonomic systems. *Self-Organization in Embedded Real-Time Systems*. Springer: New York, 2013; 151–177.
11. Fitzgerald B, Stol K-J. Continuous software engineering and beyond: trends and challenges. *First Workshop on Rapid Continuous Software Engineering (RCoSE)* co-located with ICSE’14, 2014, In Press.
12. Humphrey WS. *Managing the Software Process (Hardcover)*. Addison-Wesley Professional: Boston, MA, 1989.
13. Fitzgerald B, Stol KJ, O’Sullivan R, O’Brien D. Scaling agile methods to regulated environments: an industry case study. *Proceedings of the 2013 International Conference on Software Engineering*. IEEE Press, 2013.
14. Mc Hugh M, Cawley O, McCaffery F, Richardson I, Wang X. An agile V-model for medical device software development to overcome the challenges with plan- driven software development lifecycles. *Software Engineering in Health Care (SEHC), 2013 5th International Workshop on*, May 2013; 12–19.
15. AAMI TIR 45:2012 – Guidance on the use of agile practices in the development of medical device software, published in 2012.